

Block Ciphers

Block ciphers are a type of encryption algorithm that processes data in fixed-size chunks called blocks. They are widely used in symmetric-key cryptography (where the same key is used for encryption and decryption). Here are some examples of block ciphers, along with a brief explanation of each:

DES (Data Encryption Standard):

- **What it is:** One of the earliest symmetric encryption algorithms, developed in the 1970s.
- **Key Size:** Uses a 56-bit key (which is relatively short by today's standards).
- **Block Size:** Encrypts data in 64-bit blocks.
- **How it works:** It uses a series of complex steps (like substitution and permutation) to scramble the data.
- **Why it's outdated:** The 56-bit key is too small, making it vulnerable to brute-force attacks (where hackers try every possible key combination). It's no longer considered secure for modern applications.
- **Replacement:** AES is now the preferred choice.

AES (Advanced Encryption Standard):

<https://www.youtube.com/watch?v=O4xNJstN6E>

<https://www.youtube.com/watch?v=3MPkc-PFSRI>

- **What it is:** The most widely used symmetric encryption algorithm today, adopted by the U.S. government and many organizations.
 - **Key Sizes:** Supports 128-bit, 192-bit, or 256-bit keys. The larger the key, the stronger the encryption.
 - **Block Size:** Encrypts data in 128-bit blocks.
 - **How it works:** It uses multiple rounds of substitution, shifting, and mixing to transform the data. The number of rounds depends on the key size (10 rounds for 128-bit, 12 for 192-bit, and 14 for 256-bit).
 - **Why it's popular:** It's fast, secure, and efficient, even on devices with limited resources like smartphones.
 - **Use Cases:** Used in Wi-Fi security (WPA2/WPA3), VPNs, file encryption, and more.
-

IDEA (International Data Encryption Algorithm):

- **What it is:** A symmetric-key algorithm designed in the early 1990s as a stronger alternative to DES.
 - **Key Size:** Uses a 128-bit key, making it more secure than DES.
 - **Block Size:** Encrypts data in 64-bit blocks.
 - **How it works:** It uses a combination of XOR operations, modular arithmetic, and other mathematical operations to scramble data.
 - **Why it's less common:** While it's secure, it's not as fast or widely supported as AES. It's also patented, which limited its adoption.
 - **Use Cases:** Used in some older systems like PGP (Pretty Good Privacy) for email encryption.
-

Blowfish:

- **What it is:** A symmetric-key algorithm designed in 1993 as a fast and free alternative to DES.
 - **Key Size:** Supports variable key lengths from 32 bits up to 448 bits, making it very flexible.
 - **Block Size:** Encrypts data in 64-bit blocks.
 - **How it works:** It uses a Feistel network structure (splitting data into two halves and processing them in multiple rounds) with key-dependent S-boxes (substitution boxes) for encryption.
 - **Why it's less used today:** While it's secure and fast, it has been largely replaced by AES, which is more efficient and standardized.
 - **Use Cases:** Still used in some legacy systems and applications where flexibility in key size is needed.
-

Summary Table:

Algorithm	Key Size	Block Size	Speed	Security Level	Common Use Cases
DES	56-bit	64-bit	Slow	Weak	Legacy systems (no longer recommended)
AES	128/192/256-bit	128-bit	Fast	Very Strong	Wi-Fi, VPNs, file encryption, etc.
IDEA	128-bit	64-bit	Medium	Strong	Older email encryption (e.g., PGP)
Blowfish	32-448-bit	64-bit	Fast	Strong	Legacy systems, flexible key needs

Key Takeaways:

- **DES** is outdated and insecure due to its small key size.
- **AES** is the gold standard for modern encryption, balancing speed and security.
- **IDEA** is secure but less efficient and not widely used anymore.
- **Blowfish** is flexible and fast but has been overshadowed by AES.

AES Example

Complete AES Example: Encryption & Decryption

A Simplified Miniature Version

 SPECIFICATIONS

Parameter	Real AES	Our Mini-AES
Block Size	128 bits	16 bits
Key Size	128/192/256 bits	16 bits
State Matrix	4×4 bytes	2×2 nibbles (4-bit)
Rounds	10/12/14	2
Operations	Complex GF(256)	Simplified operations

INITIAL DATA

Plaintext:

text

1001 0110 1100 0011

Key:

text

1010 0101 1111 0000

CONVERSION TO MATRIX FORM

Plaintext Matrix (Column-Major):

text

Column 0	Column 1	
-----	-----	
1001	1100	← Row 0
0110	0011	← Row 1

$P = \begin{bmatrix} 1001, & 1100, \\ 0110, & 0011 \end{bmatrix}$

Initial Key Matrix (K_0):

text

$K_0 = \begin{bmatrix} 1010, & 1111, \\ 0101, & 0000 \end{bmatrix}$

KEY EXPANSION

S-Box Definition (4-bit):

Input	Output	Binary
0	9	0000 → 1001
1	4	0001 → 0100
2	A	0010 → 1010
3	B	0011 → 1011
4	D	0100 → 1101
5	1	0101 → 0001
6	8	0110 → 1000
7	5	0111 → 0101
8	6	1000 → 0110
9	2	1001 → 0010
A	0	1010 → 0000
B	C	1011 → 1100
C	E	1100 → 1110
D	F	1101 → 1111
E	3	1110 → 0011
F	7	1111 → 0111

Generate K_1 :

1. Take last column of K_0 : [1111, 0000]
2. Rotate up: [0000, 1111]
3. S-Box substitution:
 - 0000 → 1001
 - 1111 → 0111
 - Result: [1001, 0111]
4. XOR with first column of K_0 [1010, 0101]:

text

$$1001 \oplus 1010 = 0011$$

$$0111 \oplus 0101 = 0010$$

New column: [0011, 0010]

5. Second column of K_1 :

text

Second col of $K_0 \oplus$ new column:

$$[1111, 0000] \oplus [0011, 0010] = [1100, 0010]$$

Final K_1 :

text

$$K_1 = \begin{bmatrix} 0011, & 1100 \\ 0010, & 0010 \end{bmatrix}$$

ENCRYPTION PROCESS

ROUND 0: AddRoundKey

text

State = $P \oplus K_0$

$$\begin{bmatrix} [1001, 1100], \\ [0110, 0011] \end{bmatrix} \oplus \begin{bmatrix} [1010, 1111], \\ [0101, 0000] \end{bmatrix} = \begin{bmatrix} [0011, 0011], \\ [0011, 0011] \end{bmatrix}$$

ROUND 1:

1. SubBytes

text

All elements $0011 \rightarrow S\text{-Box}(3) = 1011$

$$\begin{bmatrix} [0011, 0011], \\ [0011, 0011] \end{bmatrix} \rightarrow \begin{bmatrix} [1011, 1011], \\ [1011, 1011] \end{bmatrix}$$

2. ShiftRows

text

Row 0: unchanged

Row 1: shift left by 1

$$\begin{bmatrix} [1011, 1011], \\ [1011, 1011] \end{bmatrix} \rightarrow \begin{bmatrix} [1011, 1011], \\ [1011, 1011] \end{bmatrix} \text{ (identical)}$$

3. MixColumns

Simple XOR-based mixing:

For each column $[a, b]$: new column = $[a \oplus b, a]$

Column 0: $[1011, 1011]$

- $a \oplus b = 1011 \oplus 1011 = 0000$
- $a = 1011$
- Result: $[0000, 1011]$

Column 1: Same $\rightarrow [0000, 1011]$

text

```
[[1011, 1011],  →  [[0000, 0000],  
 [1011, 1011]]    [1011, 1011]]
```

4. AddRoundKey (K_i)

text

```
[[0000, 0000],  ⊕  [[0011, 1100],  =  [[0011, 1100],  
 [1011, 1011]]    [0010, 0010]]    [1001, 1001]]
```


ROUND 2 (Final):

1. SubBytes

text

$0011 \rightarrow 1011$ (S-Box(3) = B)

$1100 \rightarrow 1110$ (S-Box(C) = E)

$1001 \rightarrow 0010$ (S-Box(9) = 2)

$$\begin{bmatrix} 0011, & 1100 \\ 1001, & 1001 \end{bmatrix} \rightarrow \begin{bmatrix} 1011, & 1110 \\ 0010, & 0010 \end{bmatrix}$$

2. ShiftRows

text

$$\begin{bmatrix} 1011, & 1110 \\ 0010, & 0010 \end{bmatrix} \rightarrow \begin{bmatrix} 1011, & 1110 \\ 0010, & 0010 \end{bmatrix}$$

3. AddRoundKey (K_1)

text

$$\begin{bmatrix} 1011, & 1110 \\ 0010, & 0010 \end{bmatrix} \oplus \begin{bmatrix} 0011, & 1100 \\ 0010, & 0010 \end{bmatrix} = \begin{bmatrix} 1000, & 0010 \\ 0000, & 0000 \end{bmatrix}$$

FINAL CIPHERTEXT

Convert matrix to bits (column-major):

```
text
```

```
[[1000, 0010],  
 [0000, 0000]]
```

```
Column 0: 1000 0000
```

```
Column 1: 0010 0000
```

```
CIPHERTEXT = 1000 0000 0010 0000
```

DECRYPTION PROCESS

INITIAL STATE:

text

Ciphertext: 1000 0000 0010 0000 → Matrix: $\begin{bmatrix} [1000, 0010], \\ [0000, 0000] \end{bmatrix}$

INVERSE FINAL ROUND:

1. AddRoundKey⁻¹ (K₁)

text

$\begin{bmatrix} [1000, 0010], \\ [0000, 0000] \end{bmatrix} \oplus \begin{bmatrix} [0011, 1100], \\ [0010, 0010] \end{bmatrix} = \begin{bmatrix} [1011, 1110], \\ [0010, 0010] \end{bmatrix}$

2. Inverse ShiftRows

text

Row 0: unchanged

Row 1: shift right by 1

$\begin{bmatrix} [1011, 1110], \\ [0010, 0010] \end{bmatrix} \rightarrow \begin{bmatrix} [1011, 1110], \\ [0010, 0010] \end{bmatrix}$

3. Inverse SubBytes

text

1011 (B) $\rightarrow$ 0011 (3)

1110 (E) $\rightarrow$ 1100 (C)

0010 (2) $\rightarrow$ 1001 (9)

$\begin{bmatrix} 1011, & 1110 \\ 0010, & 0010 \end{bmatrix} \rightarrow \begin{bmatrix} 0011, & 1100 \\ 1001, & 1001 \end{bmatrix}$

INVERSE ROUND 1:

1. AddRoundKey⁻¹ (K₁)

text

$\begin{bmatrix} 0011, & 1100 \\ 1001, & 1001 \end{bmatrix} \oplus \begin{bmatrix} 0011, & 1100 \\ 0010, & 0010 \end{bmatrix} = \begin{bmatrix} 0000, & 0000 \\ 1011, & 1011 \end{bmatrix}$

2. Inverse MixColumns

Inverse of our mix:

For column $\begin{bmatrix} x, & y \end{bmatrix}$, original was $\begin{bmatrix} a \oplus b, & a \end{bmatrix}$

So: $a = y$, $b = x \oplus y$

Column 0: $\begin{bmatrix} 0000, & 1011 \end{bmatrix}$

- $a = 1011$
- $b = 0000 \oplus 1011 = 1011$
- Original: $\begin{bmatrix} 1011, & 1011 \end{bmatrix}$

Column 1: Same $\rightarrow$ [1011, 1011]

text

```
[[0000, 0000],  →  [[1011, 1011],  
[1011, 1011]]      [1011, 1011]]
```

3. Inverse ShiftRows

text

```
[[1011, 1011],  →  [[1011, 1011],  
[1011, 1011]]      [1011, 1011]]
```

4. Inverse SubBytes

text

1011 (8) $\rightarrow$ 0011 (3)
All become 0011

```
[[1011, 1011],  →  [[0011, 0011],  
[1011, 1011]]      [0011, 0011]]
```

5. AddRoundKey⁻¹ (K₀)

text

```
[[0011, 0011],  ⊕  [[1010, 1111],  =  [[1001, 1100],  
[0011, 0011]]      [0101, 0000]]      [0110, 0011]]
```

✓ FINAL RESULT

Convert matrix to bits:

```
text

[[1001, 1100],
 [0110, 0011]]

Column 0: 1001 0110
Column 1: 1100 0011

RECOVERED PLAINTEXT = 1001 0110 1100 0011
```

✓ **VERIFIED:** Recovered plaintext matches original exactly!

📊 VISUAL SUMMARY

Encryption Flow:

```
text

Plaintext → AddKey( $K_0$ ) → SubBytes → ShiftRows → MixColumns → AddKey( $K_1$ )
           → SubBytes → ShiftRows → AddKey( $K_1$ ) → Ciphertext
```

Decryption Flow (Reverse):

```
text

Ciphertext → AddKey-1( $K_1$ ) → InvShiftRows → InvSubBytes
           → AddKey-1( $K_1$ ) → InvMixColumns → InvShiftRows
           → InvSubBytes → AddKey-1( $K_0$ ) → Plaintext
```

🎯 KEY TAKEAWAYS

1. **AES Structure:** SPN (Substitution-Permutation Network)
2. **All Operations Invertible:** Each step has a mathematical inverse
3. **Key Expansion:** Different round keys derived from original
4. **Confusion & Diffusion:** Achieved through SubBytes and MixColumns
5. **Symmetric:** Same key for encryption and decryption

Operation	Real AES	Our Version
SubBytes	256-byte S-Box	16-nibble S-Box
ShiftRows	(0,1,2,3) shifts	(0,1) shifts
MixColumns	GF(256) matrix mult	Simple XOR mixing
Key Expansion	Complex Rijndael	Simple rotation+XOR